

Naming Convention Utility

A small Unreal Engine editor plugin that applies consistent file naming conventions to your project assets and folders.

The plugin is designed to:

- Take a simple **DataTable** definition of naming rules.
- Convert that table into a **UNamingConventionsDataAsset**.
- Apply those rules automatically to all assets inside selected folders.
- Fix up redirectors afterwards (similar to `_“Fix Up Redirectors in Folder”_`).

> Tested with Unreal Engine 5.4 (should work with close versions as well).

1. What the Plugin Does

The **Naming Convention Utility** helps you enforce a clean, consistent naming scheme across your project.

It provides two core workflows:

1. **Convert a Naming Convention DataTable → Data Asset**

- You define rules in a `UDDataTable` that uses the row struct `FNamingConventionTableInformation`.
- The plugin converts this table into a `UNamingConventionsDataAsset` that is used at runtime for renaming.

2. **Apply Naming Conventions to a Folder**

- From the Content Browser, you choose a folder and run **“Apply Naming Conventions”**.
- The plugin renames all supported assets inside that folder (recursively) according to your rules.
- After renaming it automatically fixes up redirectors inside the same folder(s).

2. Features

- **Folder-based renaming**
 - Right-click any folder in the Content Browser.
 - Apply naming rules to all assets in that folder and all sub-folders.
- **Data-driven configuration**
 - Naming rules live in a `UNamingConventionsDataAsset`.
 - The data asset is generated from a simple `UDataTable`, so you can maintain everything in CSV / spreadsheet form.
- **Redirector cleanup**
 - After renaming, the plugin automatically calls `IAssetTools::FixupReferencers(...)` for all redirectors in the affected folders (similar to `“Fix Up Redirectors in Folder”`).
- **Supports many common asset types**
 - Static Meshes, Skeletal Meshes
 - Textures
 - Materials / Material Instances / Material Functions
 - Animations (Sequences, Montages, BlendSpaces, Composites)
 - Skeletons, PhysicsAssets
 - Blueprints, Widget Blueprints
 - Sounds (SoundWave, SoundCue)
 - Particles, Niagara Systems / Emitters
 - Worlds, Levels
 - DataTables, CurveTables, CurveFloat

3. Plugin Location & Installation

3.1 Folder layout

Place the plugin into your project (recommended):

```
``text
<ProjectRoot>/Plugins/NamingConventionUtility/
  NamingConventionUtility.uplugin
  Source/
    NamingConventionUtility/
      NamingConventionUtility.Build.cs
      NamingConventionUtility.h / .cpp
    Structs/
    Enums/
    Data/
    ...
  Content/
  Config/
  ...
```

You can also install it as an engine plugin under `Engine/Plugins`, but project-local usage is usually easier to maintain.

3.2 Enabling the plugin

1. Open your project in Unreal Editor.
2. Go to **Edit → Plugins**.
3. Search for **“Naming Convention Utility”**.
4. Enable the plugin.
5. Restart the editor if prompted.

4. Setup

4.1 Project Settings

After enabling the plugin, open:

> **Edit → Project Settings → Editor → Naming Convention Utility**

There you will find at least one key field:

- **Naming Conventions Data Asset** (`UNamingConventionsDataAsset`)
 - Reference to the data asset that holds all naming rules.
 - This asset is used when you run **“Apply Naming Conventions”** on a folder.

> If this field is not set, the plugin cannot rename anything.

4.2 Required Types

The plugin uses the following core data types:

- `FNamingConventionTableInformation` (row struct for the DataTable)
 - Fields:
 - `FileType` (`EFileTypeIdIdentifier`) – which kind of asset (StaticMesh, Texture, Blueprint, ...)
 - `ToolType` (`ENamingConventionToolType`) – what this row describes (value to remove, prefix, postfix).
 - `Value` (FString) – actual fragment (string) to use or remove.
- `UNamingConventionsDataAsset`
 - Contains an array of `FNamingConventions`.
 - Each `FNamingConventions` contains:
 - `FileTypeIdIdentifier` (`EFileTypeIdIdentifier`)
 - `NamingsToRemove` (`TArray<FString>`)
 - `PreFix` (FString)
 - `PostFix` (FString)

The plugin takes the DataTable with `FNamingConventionTableInformation` rows, aggregates all rows per `FileType`, and builds/updates the `NamingConventions` array in the data asset.

5. Creating the Naming Convention Data

Step 1 – Create the DataTable

1. In the Content Browser, create a **Data Table**.
2. As row struct choose **FNamingConventionTableInformation**.
3. Fill in rows:
 - `FileType` – choose the asset type (StaticMesh, Texture, Blueprint, ...).
 - `ToolType` – choose one of:
 - `NCTT_TOREMOVE` – a substring that should be removed from the name.
 - `NCTT_PREFIX` – the prefix that should be applied for this asset type.
 - `NCTT_POSTFIX` – the postfix that should be applied for this asset type.
 - `Value` – string fragment (e.g. `"SM_"`, `"BP_"`, `"_Mat"`, `"_LOD0"` etc.).

You can have multiple `NCTT_TOREMOVE` rows per file type and at most one prefix and postfix per type (the last one wins if there are several).

Step 2 – Convert DataTable → Data Asset

1. In the Content Browser, **right-click the DataTable** that uses `FNamingConventionTableInformation` as row struct.
2. In the context menu you should see a section **“Naming Convention Utility”**.
3. Choose **Convert NamingConvention Table to DataAsset**.
4. The plugin will:
 - Look up your `UNamingConventionsDataAsset` from **Project Settings → Editor → Naming Convention Utility**.
 - Clear its `NamingConventions` array.
 - Group all DataTable rows by `FileType` and build `FNamingConventions` entries.
 - Save the result into the data asset (marking the package as dirty).

Now your data asset contains one entry per file type with a list of fragments to remove and the final prefix / postfix to apply.

6. Applying Naming Conventions to a Folder

Once the data asset is configured, you can rename assets per folder.

1. **Right-click a folder** in the Content Browser (e.g. `/Game/Environment`).
2. Choose from the context menu:
 - **Naming Convention Utility → Apply Naming Conventions**
3. The plugin will:
 1. Scan that folder and all sub-folders using the Asset Registry.
 2. For each asset:
 - Determine its type (`EFileTypeInfo`) – StaticMesh, Texture, Blueprint, etc.
 - Look up a matching `FNamingConventions` entry in the data asset.
 - Build a new name by
 - removing all `NamingsToRemove` fragments
 - stripping existing occurrences of `PreFix` / `PostFix`
 - re-applying the configured `PreFix` at the beginning and `PostFix` at the end.
 3. If the new name is different and non-empty, schedule a rename via `FAssetRenameData`.
 4. Perform all renames in one batch using `IAssetTools::RenameAssets`.
 5. If anything was renamed:
 - Search for `UObjectRedirector` assets in the same folders.
 - Call `IAssetTools::FixupReferencers` with `ERedirectFixupMode::DeleteFixedUpRedirectors` to update references and delete redirectors.

This is roughly equivalent to:

- Renaming assets manually according to your convention.
- Running **“Fix Up Redirectors in Folder”** afterwards.

But fully automated and data-driven.

7. Supported Asset Types

The exact mapping is defined in ``ResolveFileTypeFromAsset``. Currently supported types include (but are not limited to):

- **Textures**
 - ``UTexture2D``, ``UTexture``
- **Materials**
 - ``UMaterialInstance``, ``UMaterialFunction``, ``UMaterialInterface``
- **Meshes**
 - ``UStaticMesh``, ``USkeletalMesh``
- **Animations**
 - ``UAnimSequence``, ``UAnimMontage``, ``UBlendSpace``, ``UAnimComposite``
- **Skeleton / Physics**
 - ``USkeleton``, ``UPhysicsAsset``
- **Blueprints**
 - ``UBlueprint``, ``UUserWidget``
- **Audio**
 - ``USoundWave``, ``USoundCue``
- **FX**
 - ``UParticleSystem``, ``UNiagaraSystem``, ``UNiagaraEmitter``
- **Levels / Worlds**
 - ``UWorld``, ``ULevel``
- **Data**
 - ``UDataTable``, ``UCurveTable``, ``UCurveFloat``

Everything else falls back to ``FTC_NONE`` and will be skipped.

8. Troubleshooting

8.1 No menu entry appears on DataTables

- Make sure the DataTable uses `FNamingConventionTableInformation` as row struct.
- The plugin only adds the context menu entry if at least one selected asset is such a table.
- Check that the `NamingConventionUtility` plugin is enabled and compiled for your engine version.

8.2 “Apply Naming Conventions” does nothing

- Verify that your `Naming Conventions Data Asset` is set in `Project Settings → Editor → Naming Convention Utility`.
- Open the data asset and ensure there are entries in `NamingConventions`.
 - If it's empty, re-run `“Convert NamingConvention Table to DataAsset”` on your DataTable.
- Check the Output Log for messages from the `Log_NamingConventionUtilityModule` category to see how many assets were scanned/renamed/skipped.

8.3 Redirectors are not removed

- Redirectors are only processed if at least one asset was renamed in the selected folders.
- The plugin searches `only in the folders you called the command on` (including sub-folders). If redirectors live elsewhere, run the command on those folders too or use the built-in “Fix Up Redirectors in Folder”.

9. Limitations & Notes

- The plugin is editor-only; it does not run in packaged builds.
- All changes are regular asset renames and redirector fixups – nothing special or proprietary. You can still reorganize assets manually as usual.
- The list of supported asset types can be extended easily by editing `ResolveFileTypeFromAsset` and adding corresponding enum values in `EFileTypeIdentifier`.

If you extend or modify the plugin, keep in mind that the APIs around `FAssetData`, class names and redirectors may change between engine versions (especially from 5.4 to future releases).

10. Quick Checklist

1. Enable **Naming Convention Utility** plugin.
2. Create a **DataTable** using `FNamingConventionTableInformation``.
3. Fill rows for each file type (ToRemove / Prefix / Postfix).
4. Create a **UNamingConventionsDataAsset** and assign it in **Project Settings → Editor → Naming Convention Utility**.
5. Right-click the DataTable → `Naming Convention Utility → Convert NamingConvention Table to DataAsset``.
6. Right-click any folder → `Naming Convention Utility → Apply Naming Conventions``.
7. Done – your assets are renamed and redirectors fixed up automatically.

> **Tip:** Edit your NamingConvention table outside Unreal

>

> The `DT_NamingConvention`` DataTable can be filled or updated from external tools (Excel, Google Sheets, LibreOffice, etc.) by using Unreal's **CSV** or **JSON** import pipeline:

>

> 1. In Unreal, right-click the `DT_NamingConvention`` asset and choose **Export as CSV** or **Export as JSON** to generate a template file.

> 2. Open that file in your preferred editor and modify the rows (FileType / ToolType / Value) as needed.

> 3. Save the file.

> 4. Back in Unreal, right-click `DT_NamingConvention`` and select **Reimport** (if you overwrite the existing file) or **Reimport with New File...** to pick a different CSV/JSON file.

>

> On reimport, Unreal updates the DataTable rows. You can then run **Convert NamingConvention Table → DataAsset** again to push those changes into your `UNamingConventionsDataAsset``. This workflow lets you maintain all naming rules comfortably outside the editor while still driving the plugin entirely from data.